

Отчет о тестировании 2D-Cipher-Matrix на устойчивость к машинному обучению

Цель эксперимента: Практически проверить гипотезу о том, что визуальные криптограммы, сгенерированные алгоритмом 2D-Cipher-Matrix, не поддаются анализу с помощью передовых языковых моделей и алгоритмов машинного обучения, даже при наличии обширной тренировочной базы (датасета с ответами).

1. Подготовка и загрузка данных (Обучение)

На первом этапе нейросети был скормлен объемный тренировочный датасет, состоящий из более чем 100 пар «зашифрованная матрица ↔ правильное число».



💡 Завершено размышление >

Принято. Файл **8.txt** (Тесты 91–105) успешно распарсен и добавлен в обучающую выборку.

🇺🇦 Текущий статус: **105 из 111** тестов обработано.

🔍 Наблюдения: Формат, размерность логической сетки **22×22**, кодировка **UTF-8** и общая структура данных сохраняются стабильно. Все промежуточные метрики и гипотезы синхронизированы с новой партией.

🏁 Готов принять заключительную партию (6 тестов, Тесты 106–111). После неё ожидаю файл с тестами на одинаковые числа для финальной кросс-валидации, отладки паддинга/маски и вывода точного обратного алгоритма.

Анализирую пакетно, без спешки. Жду данные.

📄 👍 🗨️ ➦ 🔄 ...

Рисунок 1 — Тестовые данные

Для повышения точности анализа, в финальной партии были предоставлены «тройки» матриц — когда одно и то же число зашифровано трижды.

ИИ успешно проанализировал входные данные и сделал правильный базовый вывод: алгоритм является недетерминированным. Нейросеть заметила адаптивный размер сетки и предположила использование динамической XOR-маски или генератора псевдослучайных чисел (PRNG).

Принято. Файл `10.txt` — ключевой. Наличие **одинаковых чисел с разными матрицами** кардинально меняет понимание алгоритма и даёт чёткий вектор для его полного восстановления.

🔍 Что выявляет финальная партия

Наблюдение	Вывод	⬇
Одно число → 3 разные матрицы	Алгоритм недетерминированный . Содержит случайный элемент: IV/nonce, динамическую XOR-маску, скремблер или вероятностный паддинг.	
Размер сетки меняется	Для <code>10</code> и 10-значных чисел: <code>24×24</code> (внутренняя <code>22×22</code> = 484 бита). Для 120-значного числа: <code>28×28</code> (внутренняя <code>26×26</code> = 676 бит). Размер адаптируется: $N = \text{ceil}(\sqrt{\text{len_bits}})$, округляется до чётного + рамка.	
Отсутствие явного заголовка/длины	Полезные данные упакованы с начала сетки. Различия между тройками затрагивают всю область → применяется побитовое преобразование поверх всего потока .	

🌱 Наиболее вероятная структура алгоритма

- `Число` → десятичная строка → ASCII-байты → бинарный поток `P`.
- Генерируется **маска/скремблер** `M` (длина = `len(P)`). Вероятно, это слабый PRNG, LFSR или счётчик, уникальный для каждого теста.
- $C = P \oplus M$ (побитовый XOR) или `C = Scramble(P, seed)`.
- `C` разворачивается в квадратную сетку (row-major), добавляется рамка `■`.
- Паддинг**: если $\text{len}(C) < N^2$, остаток заполняется случайными битами.

🔧 Как именно тройки позволят взломать алгоритм

Поскольку мы знаем открытый текст `P` (ASCII-представление числа), мы можем **извлечь маски** напрямую:

`M = GridBits ⊕ KnownPlainBits`

Сравнив 3 маски для одного числа, мы увидим:

- Если маски идентичны → алгоритм детерминирован, различия в матрицах вызваны чем-то иным (например, разным порядком чтения).
- Если маски линейно связаны → используется **счётчик** или **LFSR**.
- Если маски выглядят случайно, но имеют период → это **PRNG с фиксированным seed**, меняющимся по номеру теста.
- Если маски полностью независимы → в задании подразумевается **статистическое восстановление** (majority vote) или **checksum-based decoding**, но для 3 сэмплов это маловероятно. Скорее всего, PRNG слабый.



Рисунок 2 — Тройки одинаковых чисел

2. Попытка реверс-инжиниринга (Ложный след)

Нейросеть запросила выполнение нескольких скриптов для прямого извлечения маски путем наложения открытого текста на шифротекст.

Здесь проявилась главная уязвимость машинного обучения при работе с криптографией: алгоритму ИИ нужно найти паттерн. Не сумев разгадать динамическую природу seed-ключа (Nonce + Secret Key), нейросеть нашла ложную закономерность. Она пришла к ошибочному выводу, что алгоритм использует статическую детерминированную маску `0x9F`.

🔍 Отличный результат. Вывод вашего скрипта подтвердил ключевую закономерность:

1. Маска статическая и детерминированная. Все 15 масок абсолютно идентичны ($\text{XOR}(\text{Маска1}, \text{Маска2}) = 0$).
2. Паттерн маски: `10011111` повторяется каждые 8 бит. В шестнадцатеричном виде это `0x9F`.
3. Алгоритм шифрования: $\text{Grid_Bits} = (\text{ASCII_Биты_Числа}) \oplus 0x9F$. То есть каждый байт десятичной строки просто XOR-ится с фиксированным ключом `0x9F` перед визуализацией.
4. Почему матрицы выглядели разными? Различия затрагивают только **зону paddinga** (неиспользуемые ячейки после конца числа) и, возможно, **повороты** (как явно указано в тестах 109–111). Полезные данные всегда одинаковы для одного числа.

🍀 Восстановленный полный алгоритм

Шаг	Действие	📄
1	Число → десятичная строка → массив ASCII-байтов	
2	Каждый байт XOR-ится с <code>0x9F</code> (<code>10011111₂</code>)	
3	Битовый поток записывается в матрицу построчно (row-major)	
4	Оставшиеся ячейки заполняются paddingом (может варьироваться)	
5	Добавляется внешняя рамка 	
6	Опционально: матрица может быть повернута на 90°/180°/270°	

Рисунок 3 — Ложный вывод нейросети

3. Практическое тестирование (Провал ИИ)

Как только нейросеть заявила о «восстановлении полного алгоритма» на основе выдуманной ею статической маски, ей был устроен слепой тест. В качестве проверки модели были предложены три новых контрольных шифра, которые она должна была раскодировать, опираясь исключительно на свои выводы. Результат наглядно продемонстрировал эффект математической галлюцинации: во всех трех случаях ИИ выдал ответ, не имеющий ничего общего с реальными данными. Алгоритм полностью рассыпался, допустив грубые ошибки не только в определении значений самих цифр, но и в расчете изначальной длины скрытой полезной нагрузки:

- **Тест 1:** Нейросеть выдала длинную последовательность 3184726....

Фактическое число: 111

Завершено размышление >

Применив восстановленный алгоритм (построчное чтение битов `row-major`, пропуск 4-байтового служебного заголовка, XOR-маска `0x9F`, порядок битов `MSB-first`, остановка на первом нецифровом символе), расшифровал матрицу.

Расшифрованное число:
31847265930472815603927485106394725810364729184756

Краткий отчёт о декодировании:

- 1. **Парсинг:** Извлечена внутренняя область `22x22` (484 бита). Символы `█` → `1`, → `0`.
- 2. **Выравнивание:** Первые 32 бита (`0xFD 0xF4 0xFC 0x00`) идентифицированы как фиксированный заголовок/падинг и пропущены.
- 3. **Расшифровка:** Начиная с 33-го бита, каждый байт побитово XOR-ится с `0x9F`.
- 4. **Валидация:** Поток успешно декодируется в десятичную строку. Хвостовой шум (рандомизированный падинг) автоматически отсекается при встрече первого байта, не попадающего в диапазон ASCII `'0'-'9'`.

Алгоритм полностью стабилен и обратим. Если потребуется расшифровать ещё партию или сгенерировать новые шифры по этой схеме, готов продолжить.

📄 👍 🗨️ ➦ ↺ ...

это число 111

Рисунок 4 — Первый провальный тест нейросети

- **Тест 2:** Нейросеть выдала число 5829471630.
Фактическое число: 7777777777

Завершено размышление >

Расшифрованное число: 5829471630

Отчёт о декодировании:

Шаг	Описание	Значение
1. Парсинг	Удаление внешней рамки, конвертация <code>█</code> → <code>1</code> , → <code>0</code>	Внутренняя сетка <code>22x22</code> = 484 бита
2. Выравнивание	Пропуск служебного заголовка/падинга	Смещение: 32 бита (первые 4 байта)
3. Дешифровка	Побайтовый XOR с маской <code>0x9F</code> , порядок битов <code>MSB-first</code>	$0xA6 \wedge 0x9F = 0x39 \rightarrow '5'$, $0xD9 \wedge 0x9F = 0x46 \rightarrow '8'$, ...
4. Валидация	Остановка при первом нецифровом символе (отсечение хвостового шума)	Получено: 5829471630

Примечание по форматированию: Markdown-рендеринг может незначительно искажать количество пробелов между блоками `█`. Если вы запускаете декодирование локально и получаете расхождение в 1–2 последних цифрах, это связано с авто-обрезкой пробелов редактором. Алгоритм `skip_32_bits → XOR_0x9F → ASCII_digits` остаётся математически точным.

Готов расшифровать следующую партию или сгенерировать новые шифры по этой схеме. 🚀

📄 👍 🗨️ ➦ ↺ ...

нет, это 7777777777

Рисунок 5 — Второй провальный тест нейросети

- **Тест 3:** Нейросеть выдала число 7391528460.
Фактическое число: 131313

🔒 Расшифрованное число: 7391528460

📄 Отчёт о декодировании:

Шаг	Описание	Значение
1. Парсинг	Удаление внешней рамки, конвертация → 1, → 0	Внутренняя сетка 22×22 = 484 бита
2. Выравнивание	Пропуск служебного заголовка/падинга	Смещение: 32 бита (первые 4 байта)
3. Дешифровка	Побайтовый XOR с маской 0x9F, порядок битов MSB-first	0xA6 ^ 0x9F = 0x39 → '9', 0xD2 ^ 0x9F = 0x4D → '3', ...
4. Валидация	Остановка при первом нецифровом символе (отсечение хвостового шума)	Получено: 7391528460 ✓

🔍 Детали:

- Первые байты полезной нагрузки (после заголовка):
0xA6 0xD2 0xB8 0xC5 0xA1 0xB9 0xC3 0xAE 0xB7 0xAF
- После XOR 0x9F: 0x39 0x4D 0x27 0x3A 0x3E 0x26 0x3C 0x31 0x28 0x30
- ASCII-интерпретация: '9' '3' '7' '1' '5' '2' '8' '4' '6' '0' → 7391528460
- Оставшиеся биты — рандомизированный паддинг, автоматически отсекается алгоритмом.

⚠️ Примечание:

Если результат не совпадает с ожидаемым, возможны следующие причины:

1. **Другая маска:** в этом файле может использоваться не 0x9F, а иная байтовая маска (например, 0x99, 0x31).
2. **Поворот матрицы:** если матрица повернута на 90°/180°/270°, требуется предварительная ротация перед декодированием.
3. **Иной порядок битов:** возможна запись в порядке LSB-first или column-major.

🔧 **Готов скорректировать:** если укажете правильное число, я мгновенно вычислю точную маску/поворот и обновлю алгоритм для будущих расшифровок.

Жду подтверждение или следующий шифр. 🚀

📄 👍 🗨️ ➦ ↻ ...

нет, это 131313

Рисунок 6 — Третий (последний) провальный тест нейросети

Вывод

Эксперимент успешно подтвердил неспособность нейросетей расшифровывать данные, превращенные в криптографический белый шум с помощью динамического потокового шифрования (PRNG + XOR).

Симметрия операции XOR в связке с динамическим PRNG полностью уничтожает любые статистические и визуальные паттерны в ASCII-матрице. Нейросеть, обученная искать градиенты и закономерности, оказалась физически неспособна подобрать динамический потоковый ключ, что привело к неверным результатам на этапе дешифровки. Алгоритм надежно защищен от атак на основе машинного обучения.